

5. Python för Systemadministration

Arseniy Buzyan

Från operativt problem till återanvändbart verktyg



OOPS
Operations Optimization

Python-verktyg för Linux-administration som löser verkliga supportärenden med automation.

PEXA

Automatiserar granskning av lösenordsförfallodatum tvärs över flera Linux-servrar.

STAyzer

Datadriven logghantering och analys för storskaliga IT-miljöer.

Svenska arbetsgivare kräver Python-kunskaper för att effektivisera drift och sluta utföra repetitiva uppgiter manuellt.

Varför Python för Systemadministration?

- Python är "limmet" som kopplar ihop olika system och verktyg.
- PowerShell är perfekt för Windows. Python är perfekt för att:

Användningsområde	**Varför Python?**
Orkestrering	Styr andra verktyg (Terraform, Ansible, PowerShell)
Verifiering filkontroll)	Kontrollera att system fungerar (HTTP-anrop,
Datahantering	Läsa/skriva JSON, YAML, CSV, XML
Rapportering	Skapa HTML-rapporter, diagram, e-post
Integration	Koppla ihop API:er, databaser, molntjänster

Från Verkligheten – Python i Svenska IT-miljöer

- Python används för att automatisera och förenkla IT-drift i svenska företag.

- Varför detta är viktigt: Svenska arbetsgivare efterfrågar Python-kunskaper för att effektivisera IT-drift och automatisera repetitiva uppgifter. Varje OOPS-projekt börjar från ett verkligt operativt problem och utvecklas till ett återanvändbart kommandoradsverktyg.

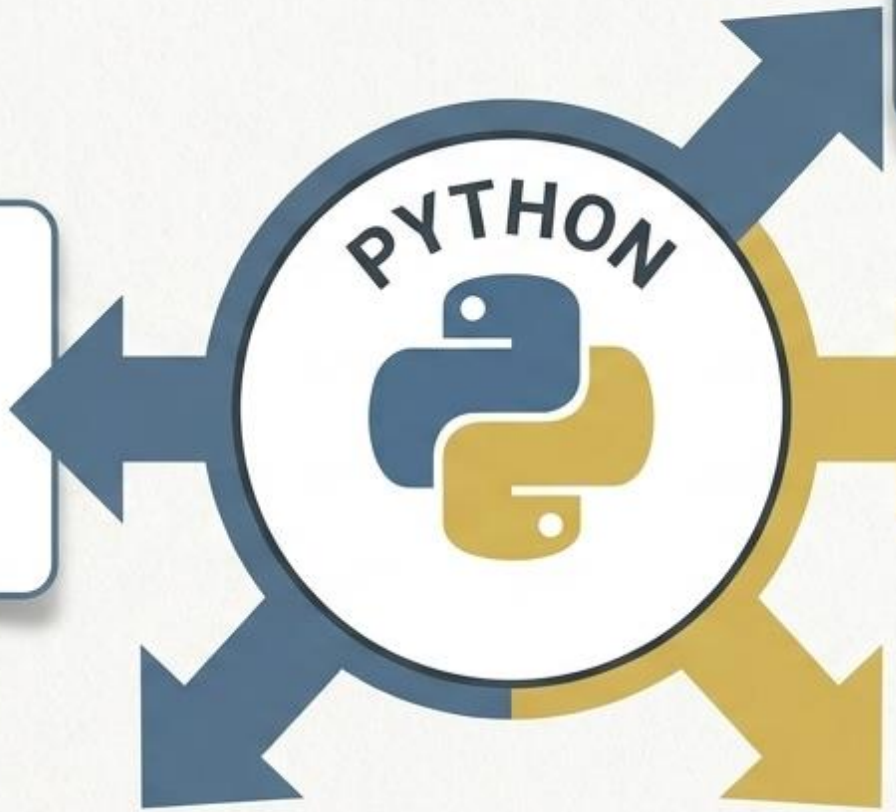
Python fungerar som limmet mellan systemen

Orkestrering

Styr andra verktyg (Terraform, Ansible, PowerShell).

Verifiering

Kontrollerar att system fungerar (HTTP-anrop, filkontroll).



Integration & Data

API:er, databaser, molntjänster. Läser och skriver JSON, YAML, CSV, XML.

Rapportering

Skapar HTML-rapporter, diagram och e-postutskick.

Rätt verktyg för rätt uppgift

	PowerShell	Python
Styrka	Windows-administration.	Flerplattformsarbete, analys, API:er.
Datahantering	Objekt-baserad pipeline. `$data[0].metadata.name`	Text-baserad / Dictionaries. `json.loads()`
Syntax	Verb-Substantiv (cmdlets).	Lättläst, ren kod.

Exempel: API-anrop

```powershell
$data = Invoke-RestMethod -Uri $url
```
(Objekt hanteras automatiskt)

```python
data = requests.get(url).json()
```
(Kräver explicit JSON-tolkning)

Exempel – bearbeta API-svar

Val av verktyg: Välj PowerShell för Windows-centrerad automatisering och komplex objektstruktur. Välj Python för flerplattformsarbete, komplex datahantering och API-integration.

Uppgift	PowerShell	Python
Hämta data	$data = Invoke-RestMethod -Uri $url	data = requests.get(url).json()
Access property	$data.items[0].metadata.name	data['items'][0]['metadata']['name']
Filtrera	$data | Where-Object {…}	Listförståelse eller filter()

Vad är JSON och YAML? – Konfigurationsformat i IT

Båda är textbaserade format för att lagra data som människor och datorer kan läsa.



Språket systemen talar: Konfigurationsformat i IT

JSON

```
{
  "server": "WEB01",
  "port": 80,
  "environment": "production"
}
```

Strikt, mer parenteser.
Inga kommentarer tillåtna.

YAML

```
server: WEB01
port: 80
environment: production
# Kommentarer stöds!
```

Renare syntax, lättläst för människor.

Standard för: API:er, Molntjänster, Terraform	Standard för: Kubernetes, Ansible, Docker Compose

Varför används de i IT? Båda formaten används för att lagra konfiguration – precis som du sparar inställningar i Windows, sparar du konfiguration i JSON/YAML. JSON är standard för webb-API:er och molntjänster . YAML är standard för Kubernetes, Docker Compose och Ansible, eftersom det är lättare för människor att läsa.

Jämförelse

Egenskap	JSON	YAML
Läsbarhet	Svårare för människor (mycket parenteser)	Lättare för människor (renare syntax)
Kommentarer	Stöds inte	Stöds (# kommentar)
Användning	API:er, Terraform, molntjänster	Ansible, Kubernetes, Docker Compose
Strikt syntax	Ja – inga kommentarer, måste ha korrekta citattecken	Mer flexibel

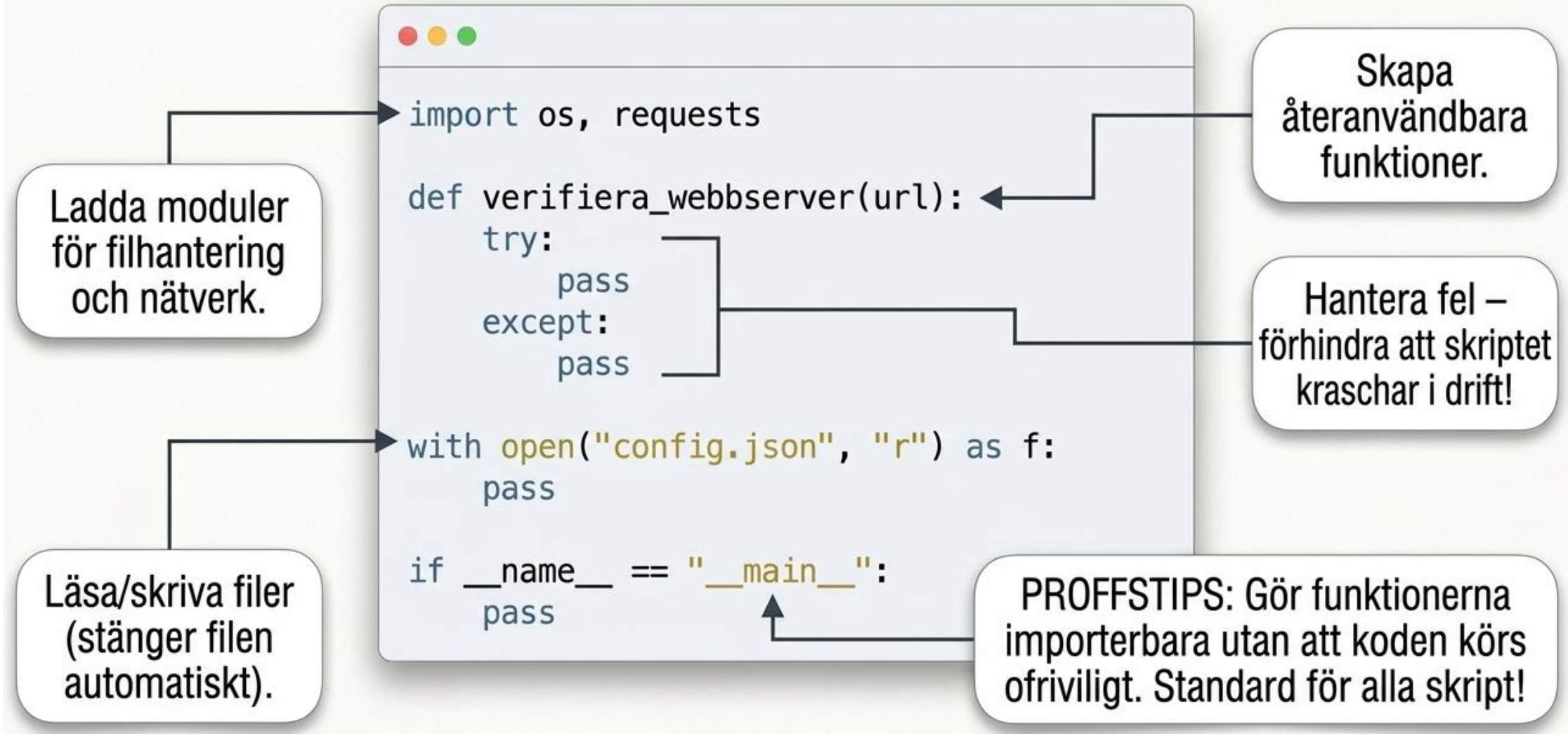
Python Grundkoncept – För Systemadministratörer

Du behöver inte bli Python-expert – bara kunna skriva funktionella skript.

Viktiga koncept för systemadministration:

Koncept	Varför behövs det?	Exempel
import	Ladda moduler för filhantering, nätverk, etc.	import os, subprocess, requests
def	Skapa återanvändbara funktioner	def verifiera_webbserver(url):
try/except	Hantera fel – skriptet ska inte krascha!	try: requests.get(url) except: ...
with open()	Läsa/skriva filer (stänger automatiskt)	with open("config.json", "r") as f:
if __name__ == "__main__"	Kör skriptet direkt eller importera som modul	Standard för alla skript!

Minimum Viable Python: Kärnkoncept för Administratörer



Exempel – komplett skriptstruktur

Proffstips: Använd if __name__ == "__main__" – då kan du importera funktioner från ditt skript senare utan att koden körs automatiskt.

python

```python
import requests

def verifiera_webbserver(url):
    try:
        response = requests.get(url, timeout=5)
        return response.status_code == 200
    except:
        return False

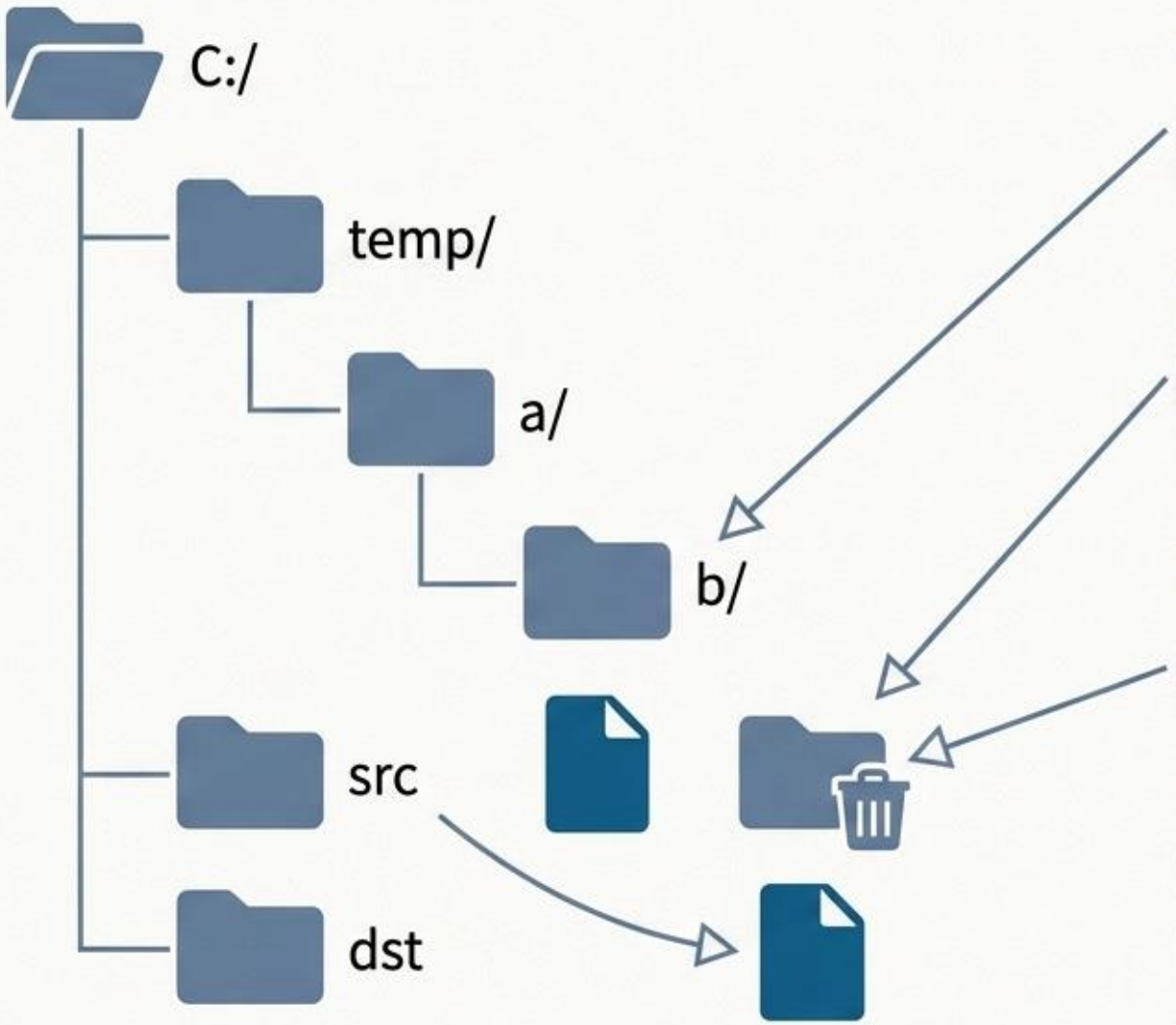
if __name__ == "__main__":
    if verifiera_webbserver("http://localhost"):
        print("Webbserver OK!")
    else:
        print("Webbserver EJ OK!")
```

Varje modul har ett specifikt syfte – använd rätt verktyg för rätt uppgift!

Modul	Syfte	Exempel
os	Fil- och mapphantering, miljövariabler	os.makedirs(), os.path.exists()
shutil	Avancerad filhantering (kopiera, flytta, ta bort)	shutil.copy(), shutil.rmtree()
subprocess	Kör systemkommandon (PowerShell, Bash)	subprocess.run(["ls", "-la"])
json	Läsa/skriva JSON-filer	json.load(), json.dump()
yaml	Läsa/skriva YAML-filer	yaml.safe_load(), yaml.dump()
requests	HTTP-anrop (API:er, webbservrar)	requests.get("http://localhost")
logging	Strukturerad loggning med nivåer	logging.info("Skriptet startades")

Varför detta är viktigt: I professionella miljöer används dessa moduler dagligen för att automatisera allt från backup-validering till API-integration. `os.path` har historiskt varit standard för filsökvägar, men sedan Python 3.4 rekommenderas `pathlib` som ett renare alternativ . `shutil` används för hög-nivå filoperationer som att kopiera hela mappträd .

Filhantering på Djupet: Trädet och Verktygen



Action 1

```
os.makedirs("C:/temp/a/b", exist_ok=True)
```

Action 2

```
shutil.copytree("src", "dst")
```

(Kopierar alla behörigheter och allt innehåll)

Action 3

```
shutil.rmtree()
```

PROFFSTIPS: Från Python 3.4+ rekommenderas `pathlib` som ett renare alternativ till `os.path`.

VARNING: Undvik `os.chdir()` (ändra arbetsmapp), det leder till oförutsedda buggar i automationsskript.

os-modulen – Grunden för Filhantering

os-modulen är grunden för all fil- och mapphantering i Python.

Funktion	Vad gör den?	Exempel
os.listdir()	Lista filer i en mapp	os.listdir("C:/temp")
os.path.join()	Slå ihop sökvägar (plattformsoberoende!)	os.path.join("C:/temp", "file.txt")
os.path.exists()	Kontrollera om fil/mapp finns	os.path.exists("C:/temp/file.txt")
os.makedirs()	Skapa mappar (rekursivt!)	os.makedirs("C:/temp/a/b/c", exist_ok=True)
os.walk()	Gå igenom alla filer i en mappträd	for root, dirs, files in os.walk("C:/temp"):

Exempel – skapa en mappstruktur

Varning: Att ändra arbetsmapp med `os.chdir()` kan leda till problem i skript. Undvik det om möjligt eller återställ till ursprunglig mapp.

python

```python
import os

mappar = ["C:/Projekt", "C:/Projekt/Logs", "C:/Projekt/Config"]
for mapp in mappar:
    if not os.path.exists(mapp):
        os.makedirs(mapp, exist_ok=True)
        print(f"Skapade: {mapp}")
```

shutil-modulen – Avancerad Filhantering

shutil = "shell utilities" – kopiera, flytta och ta bort filer enkelt.

Funktion	Vad gör den?	Exempel
shutil.copy()	Kopiera fil	shutil.copy("src.txt", "dst.txt")
shutil.move()	Flytta/byt namn på fil	shutil.move("old.txt", "new.txt")
shutil.rmtree()	Ta bort mapp med ALLT innehåll	shutil.rmtree("C:/temp/gammal_mapp")
shutil.copytree()	Kopiera hela mappträd	shutil.copytree("src", "dst")
shutil.diskusage()	Visa diskutrymme	total, used, free = shutil.diskusage("/")

Exempel – backup av en mapp

Proffstips: `shutil.copy()` kopierar också filens behörigheter, medan `shutil.copyfile()` bara kopierar innehållet.

```python
import shutil
import os
from datetime import datetime

källa = "C:/ViktigaFiler"
datum = datetime.now().strftime("%Y%m%d")
mål = f"C:/Backup/viktiga_filer_{datum}"

if not os.path.exists(mål):
    shutil.copytree(källa, mål)
    print(f"Backup skapad: {mål}")
else:
    print(f"Backup finns redan: {mål}")
```

subprocess – Kör Externa Kommandon

subprocess låter dig köra PowerShell, Bash eller andra program från Python.

```python
import subprocess

resultat = subprocess.run(
    ["powershell", "-Command", "Get-Service"],
    capture_output=True,
    text=True
)

print(resultat.stdout)  # Visa utdata
```

Viktiga parametrar

Säkerhetsvarning: Använd ALDRIG `shell=True` med användarinput – det leder till shell injection . Använd alltid list-format istället: `["ls", "-la"]` är säkrare än `"ls -la"` med `shell=True`. Enligt Python-experter är `subprocess` det enda språket som verkligen kan litas på för systemadministration tack vare säkra standardinställningar med `shell=False` .

Parameter	Vad gör den?	Exempel
capture_output=True	Fångar stdout och stderr	subprocess.run(["ls"], capture_output=True)
text=True	Returnerar text istället för bytes	subprocess.run(["ls"], text=True)
check=True	Kastar fel om kommandot misslyckas	subprocess.run(["ls"], check=True)
cwd	Ange arbetsmapp	subprocess.run(["ls"], cwd="/tmp")

Persisterande PowerShell-sessioner – Varför Det Är Viktigt

Vanliga `subprocess.run()` anrop startar en NY PowerShell-process varje gång – detta är ineffektivt!

Problem med `subprocess.run()`:

- Varje kommando startar en ny `pwsh`-process
- Autentisering (t.ex. Azure, Exchange Online) förloras mellan kommandon
- Variabler och tillstånd förloras mellan anrop
- Långsamt – processstart är dyrt

Lösning – Persisterande session med py-pwsh-session:

```
python
from py_powershell import PowerShellSession

with PowerShellSession() as pwsh:
    # Autentisera en gång – sessionen förblir inloggad
    pwsh.execute("Connect-AzAccount")

    # Alla efterföljande kommandon använder samma autentisering
    vms = pwsh.execute("Get-AzVM | Select-Object Name", json_parse=True)
    print(vms)
```

Jämförelse

När ska man använda vilken metod? Använd `subprocess.run()` för enkla engångskommandon. Använd persisterande session när du arbetar med flera kommandon som behöver samma autentisering (Azure, Exchange Online, Microsoft Graph) .

Aspekt	subprocess.run()	**Persisterande session**
Processstart	Varje kommando	En gång
Autentisering	Förloras efter varje kommando	Består över flera kommandon
Variabler/tillstånd	Förloras mellan anrop	Bevaras
Modulfrågor	Misslyckas (ingen auth-kontext)	Fungerar (samma session)
Prestanda	Långsam (process-overhead)	Snabb (återanvänd process)

JSON – Hantera Konfigurationsfiler

JSON är standardformatet för konfiguration i moderna IT-system (API:er, molntjänster, Terraform).

Läsa JSON från fil:

python

```python
import json

with open("config.json", "r") as f:
    config = json.load(f)

print(f"Server: {config['server_name']}")
print(f"Port: {config['port']}")
```

Skriva JSON till fil:

python

```python
import json

data = {"server": "WEB01", "port": 80}
with open("config.json", "w") as f:
    json.dump(data, f, indent=4)  # indent=4 gör filen läsbar
```

Viktiga funktioner

Funktion	Vad gör den?	Exempel
json.load()	Läs JSON från fil	data = json.load(f)
json.dump()	Skriv JSON till fil	json.dump(data, f, indent=4)
json.loads()	Tolka JSON-sträng	data = json.loads('{"key":"value"}')
json.dumps()	Konvertera till JSON-sträng	json.dumps(data, indent=4)

Proffstips: Använd `indent=4` för att göra JSON-filer läsbara för människor – viktigt för felsökning i produktion. Python-biblioteket `anyconfig` kan ladda konfiguration från flera format (JSON, YAML, XML) och stöder avancerade funktioner som schemavalidering .

YAML – Hantera Ansible och Kubernetes-konfiguration



YAML är standardformatet för Ansible-playbooks, Kubernetes manifest och Docker Compose.

Installation:

bash

```
pip install pyyaml
```

Läsa YAML från fil:

python

```
import yaml

with open("config.yaml", "r") as f:
    config = yaml.safe_load(f)  # safe_load är säkrare än load

print(f"Server: {config['server_name']}")
```

Skriva YAML till fil:

python

```
import yaml

data = {
    "server_name": "WEB01",
    "port": 80,
    "features": ["Web-Server", "Web-WebSockets"]
}
```

Jämförelse JSON vs YAML

Varför YAML i Kubernetes och Ansible? YAML är mer läsbart för människor, stöder kommentarer (#), och har mindre syntaktiskt brus än JSON – vilket är viktigt när du hanterar komplexa konfigurationer .

json

```
// JSON – mer parenteser, svårare för människor
{ "server": "WEB01", "environment":
"production" }
```

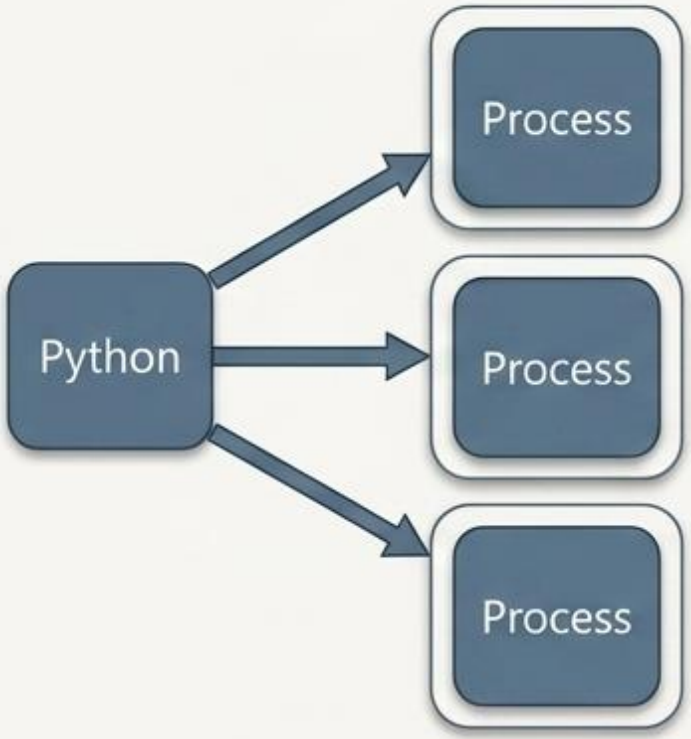
yaml

```
# YAML – renare, lättare för människor server:
WEB01 environment: production
```

Skalbarhet: Persisterande PowerShell-sessioner

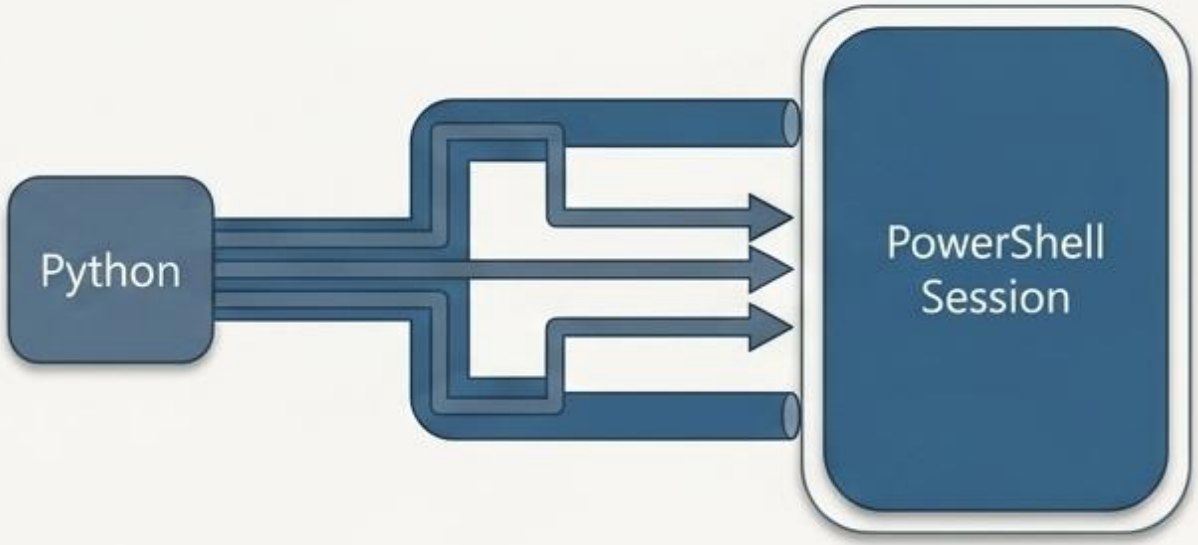
Före / Efter

Vanliga Anrop (Ineffektivt)



- Varje kommando startar en NY process.
- Långsamt och resurskrävande.
- Azure/Exchange-autentisering förloras direkt.
- Variabler glöms bort.

Persisterande Session




```
with PowerShellSession() as pwsh:
    pwsh.execute("Connect-AzAccount")
```

- Autentisera en gång. Sessionen förblir inloggad.
- Tillstånd bevaras mellan kommandon.
- Prestandan maximeras.

Orkestrering – Anropa PowerShell från Python

Python kan styra PowerShell för att utföra Windows-specifika uppgifter.

- **Enkelt exempel – kör PowerShell-kommando:**
- python

```
import subprocess

resultat = subprocess.run(
    ["powershell", "-Command", "Get-Service | Select-Object -First 3"],
    capture_output=True,
    text=True
)
print(resultat.stdout)
```

- **Exempel – köra PowerShell-skriptfil:**
- python

```
import subprocess

resultat = subprocess.run(
    ["powershell", "-File", "C:/Scripts/install_iis.ps1"],
    capture_output=True,
    text=True
```

Persisterande session – för flera kommandon

När använder man vilken metod? Använd `subprocess.run()` för enkla engångskommandon. Använd `py-pwsh-session` eller liknande bibliotek när du behöver autentisering eller tillstånd över flera kommandon .

- python

```python
from py_powershell import PowerShellSession

with PowerShellSession() as pwsh:
    # Sätt en variabel i sessionen
    pwsh.execute("$myList = @()")

    # Ackumulera data över flera kommandon
    pwsh.execute("$myList += 'item1'")
    pwsh.execute("$myList += 'item2'")
    pwsh.execute("$myList += 'item3'")
```

Workshop – Verifiera Webbserver och Läsa JSON

- Mål: Skapa ett Python-skript som läser konfiguration från JSON och verifierar webbserver.
- Diskussionsfråga: Varför är det viktigt att läsa konfiguration från en JSON-fil istället för att hårdkoda URL:en? Hur påverkar detta skriptets återanvändbarhet?

Syntes: Arkitekturen för ett Automations-skript

Data - config.json

```
{
  "web_server_url": "http://localhost",
  "timeout": 5
}
```

PROFFSTIPS: Använd `indent=4` vid `json.dump()` för att göra konfigurationen felsökningsbar för människor i produktion.

Beroenden - Terminal

```
> pip install requests
```

Hanterar extern kraft för HTTP-nätverksåtkomst.

Vi ska nu knyta ihop JSON-konfiguration, extern nätverksåtkomst och felhantering i ett enda robust skript.

Verktyget i Verkligheten (verify.py)

verify.py

```python
import json, requests

def load_config(path="config.json"):
    with open(path, "r") as f:
        return json.load(f)

def check_server(url, timeout=5):
    try:
        response = requests.get(url, timeout=timeout)
        return response.status_code == 200
    except:
        return False

if __name__ == "__main__":
    config = load_config()
    url = config["web_server_url"]
    timeout = config.get("timeout", 5)

    if check_server(url, timeout):
        print(f"Server OK: {url}")
```

Konfiguration: Data och logik är separerade. Läser externt.

Robusthet: Hanterar nätverksfel utan att krascha fult i terminalen.

Fallback: Hämtar timeout säkert, sätter 5 som standard om nyckeln saknas.

Main-block: Vår MVP-regel i aktion. Gör funktionerna återanvändbara.



Steg 1: Skapa `config.json`:

```json
{
  "web_server_url": "http://localhost",
  "timeout": 5
}
```

Steg 2: Installera requests:

```bash
pip install requests
```

Steg 3: Skapa `verify.py`:

```python
import json
import requests

def load_config(path="config.json"):
    with open(path, "r") as f:
        return json.load(f)

def check_server(url, timeout=5):
    try:
        response = requests.get(url, timeout=timeout)
        return response.status_code == 200
    except:
        return False

if __name__ == "__main__":
    config = load_config()
    url = config["web_server_url"]
    timeout = config.get("timeout", 5)

    if check_server(url, timeout):
        print(f"✅ Server OK: {url}")
    else:
        print(f"❌ Server EJ OK: {url}")
```

Steg 4: Kör skriptet:

```bash
python verify.py
```

Checklista för Produktion & Best Practices

Arkitektur & Säkerhet

- **Idempotens**: Skriptet MÅSTE tåla att köras flera gånger utan att förstöra systemtillståndet.
- **Säker Exekvering**: Undvik shell=True. Skicka alltid argument som listor till subprocess.
- **Persistens i Molnet**: Använd persisterande PowerShell-sessioner vid sekventiella Azure/Exchange-kommandon.

Modul-Lathund

`os / pathlib`	Sökvägar och mappar.
`shutil`	Kopiera/radera hela filträd.
`json`	API:er och Terraform-config.
`yaml`	Kubernetes och Ansible.
`requests`	Externa system och validering.



Sammanfattning

os – fil- och mapphantering, miljövariabler

shutil – avancerad filhantering (kopiera, flytta, ta bort)

subprocess – kör PowerShell, Bash och andra program

json – läsa/skriva konfigurationsfiler (API:er, molntjänster)

yaml – hantera Ansible, Kubernetes, Docker Compose-konfiguration

requests – testa webbservrar och API:er

Skillnaderna mellan JSON och YAML:

JSON – strikt, används i API:er och Terraform

YAML – mer läsbart, stöder kommentarer, används i Ansible och Kubernetes

Persisterande PowerShell-sessioner:

Viktigt för autentisering över flera kommandon

py-pwsh-session möjliggör Azure-autentisering och tillståndshantering

Viktiga koncept:

Idempotens – skriptet ska tåla att köras flera gånger

Säkerhet – undvik shell injection, använd shell=False

Quiz:

https://forms.gle/9EsJRfZGR4Cy6rnM9